EE3-06
EE9AC5

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2012

MSc and EEE/ISE PART III/IV: MEng, BEng and ACGI

VHDL AND LOGIC SYNTHESIS

Thursday, 3 May 2:30 pm

Time allowed: 2:00 hours

There are TWO questions on this paper.

Answer BOTH questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : T.J.W. Clarke
Second Marker(s) : G.A. Constantinides

© Imperial College London

Special Information for Invigilators: none.

Information for Candidates

VHDL language reference can be found in the booklet VHDL Exam Notes.

Unless otherwise specified assume VHDL 1993 compiler.

All packages that must be explicitly referenced with USE, e.g. IEEE.numeric_std, must be explicitly noted in each answer: semantically correct LIBRARY and USE statements may be omitted where this is done.

© Imperial College London

The Questions

All questions are compulsory

1.

a) Write a synthesisable architecture for entity *counter* in Figure 1.1 which implements an unsigned 100 bit down counter such that output q has count sequence:

$0, p-1, p-2, ..., 3, 2, 1, 0, p-1, ...$

Where p is the unsigned number on port p. Your counter must use a *negative* edge triggered clock *clk* and reset synchronously to 0 when *reset* is logic 1.

[6]

b) Draw a simulation waveform diagram of the signals $v(0)$ - $v(3)$ from architecture TEST in Figure 1.2. Your diagram must display time from 0 to 35 ns and signals in the order shown in Figure 1.3. Dimension your diagram with the simulation time and simulation delta of each signal transition.

[6]

c) Write a synthesisable architecture for entity *fsm* in Figure 1.4, with the transitions and outputs as specified in Figures 1.5 and 1.6.

[8]

```
ENTITY counter  IS
   PORT(    clk, reset: IN std_logic;
            p: IN std_logic_vector(99 DOWNTO 0);
            q: OUT std_logic_vector(99 DOWNTO 0) );
END counter;
```

Figure 1.1

```
ARCHITECTURE test of xx IS
    SIGNAL nclk, clk1, y, z: std_logic;
    SIGNAL clk: std_logic := '1';
    SIGNAL x: std_logic := '0';
    SIGNAL v: std_logic_vector(4 DOWNTO 0);
BEGIN

    PCLK: PROCESS
    BEGIN
        clk <= '1';
        WAIT FOR 10 ns;
        clk <= '0';
        WAIT FOR 5 ns;
    END PROCESS PCLK;

    nclk <= not clk;

    clk1 <= not nclk;

    z <= clk1 xor clk;

P1: PROCESS
    BEGIN
        WAIT UNTIL nclk'EVENT and nclk = '1';
        x <= not x;
    END PROCESS P1;


    P2: PROCESS
    BEGIN
        WAIT UNTIL clk'EVENT and clk = '1';
        y <= x
    END PROCESS P2;


    v <= ( z, y, x, nclk);


END ARCHITECTURE test;
```

Figure 1.2

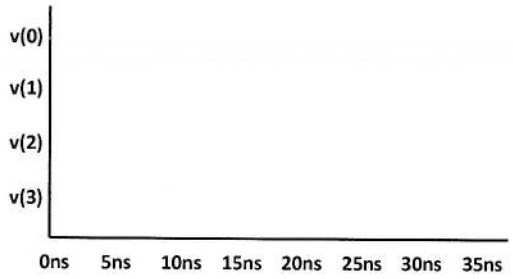


Figure 1.3

```
ENTITY fsm IS
    PORT(
        clk, reset: In std_logic;
        p,q: IN std_logic_vector(15 DOWNTO 0);
        r,w: IN std_logic_vector(31 DOWNTO 0);
        done, match1, match2: OUT: std_logic
    );
END ENTITY fsm;
```

Figure 1.4

state	next state	condition	notes
A	B	$p > r$	Comparison on signed values of p,r
A	A	$p \leq r$	
B	C	$q = w+1$	Comparison on unsigned values of q,w
B	B	$q \neq w+1$	
C	A	n/a	

NB: if *reset* = '1' the next state must be A independently of the above table

Figure 1.5 FSM transitions

state	done	match1	match2
A	0	q = p	1
B	0	0	1
C	not reset	0	0

Figure 1.6 FSM outputs: done, match1 and matchc2

© Imperial College London

2. The entity *compute* and architecture *arch* in Figures 2.1 and 2.2 generate output *y* from inputs *clk x*, *n*. Part a below concerns the architecture *arch*. Parts b and c do not depend on *arch* and are concerned only with the usage of entity *compute*.

a) Identify the three potential coding problems in architecture *arch*, which is intended to be synthesisable, indicating in each case what would be the possible issues in pre-synthesis simulation and/or synthesis without correction. You do not need to correct the code.

[6]

b) The entity *compute* is instantiated in architecture *struct1* as in Figure 2.3. Write a substitute entity *compute1* for *arch*, and make corresponding changes to the instances I1 and I2 of *struct1*, which ensure that all constants in *arch* will be synthesised as constants while preserving the functionality of *struct1*.

[4]

c) Write an architecture *power_of_compute* for entity *power* in Figure 2.4 which instantiates *R* copies of *compute* (not *compute1*) connected as shown in Figure 2.5. Note that in Figure 2.5 *R* denotes an integer constant contained in a package *power_package* used by the *power_of_compute* architecture.

[8]

d) For your implementation, state whether it would be possible to use *compute1* instead of *compute*, with appropriate changes to *power_of_compute*, giving reasons for your answer.

[2]

```
ENTITY compute IS
PORT(
    n: IN INTEGER;
    x: IN std_logic_vector( 7 DOWNTO 0);
    y: OUT std_logic_vector(7 DOWNTO 0);
    clk, reset: IN std_logic );
END ENTITY compute;
```

Figure 2.1 *Compute* entity

```
ARCHITECTURE arch OF compute IS

    SIGNAL y1: std_logic_vector( 7 DOWNTO 0);

BEGIN

        P0: PROCESS(reset)
        BEGIN
            IF reset = '1' THEN
                y1 <= (OTHERS=>'0');
            END IF;
        END PROCESS P0;

        P1: PROCESS(x)
        BEGIN
            IF reset = '0' THEN
                y1 <= x;
            END IF;
        END PROCESS P1;

        P2: PROCESS
        BEGIN
            WAIT UNTIL clk'EVENT and clk='1';
            y <= std_logic_vector( unsigned(y1)
                    + resize( n* unsigned(x), 8));
        END PROCESS P2;

END ARCHITECTURE arch;
```

Figure 2.2 *Arch* architecture

```
ARCHITECTURE struct1 OF xx IS
    SIGNAL a, b, c    : std_logic_vector(7 DOWNTO 0);
    SIGNAL clk, reset : std_logic;
BEGIN
    I1 : ENTITY compute PORT MAP (n => 2, x => a, y => b, clk => clk, reset => reset);
    I2 : ENTITY compute PORT MAP (n => 3, x => b, y => c, clk => clk, reset => reset);
END ARCHITECTURE struct1;
```

Figure 2.3 *Struct1* architecture

```
ENTITY power IS
PORT(
    a: IN std_logic_vector( 7 DOWNTO 0);
    b: OUT std_logic_vector(7 DOWNTO 0);
    clk, reset: IN std_logic );
END ENTITY power;
```

Figure 2.4 *Power* entity

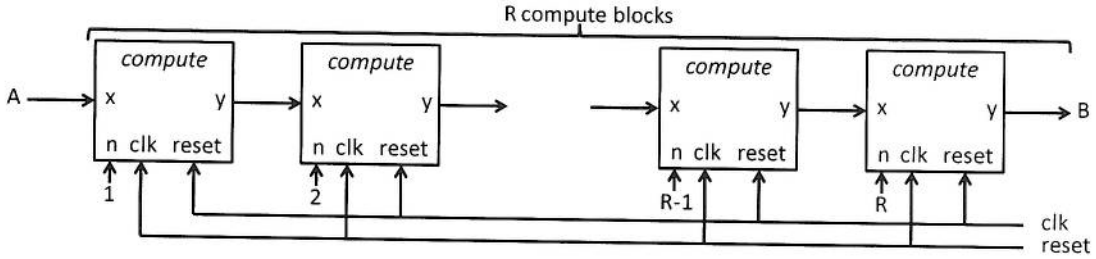


Figure 2.5 Connections of *compute* inside *power_of_compute*

© Imperial College London

VHDL EXAM NOTES

VHDL Sequential (Behavioural) Statements - inside PROCESS body

Meta-language	
[]	*optional part*
*	*0 or more repetitions*
{}	*grouping brackets*

- See also the dataflow statements which can also occur inside a PROCESS - previous slide
- Sequential statements (except WAIT) take zero simulation time to execute

```
variable := value ; -- variable assignment
NULL; -- empty statement
WAIT [ ON signal ]  [ UNTIL condition ] ;
WAIT FOR time ;

IF condition THEN statements
[ ELSIF condition THEN statements ]
[ ELSE statements ]
END IF ;

FOR var IN range LOOP
   for-statements
END LOOP;

CASE var IS
    WHEN case1 => statements
    [ WHEN case2 => statements ]
    [ WHEN OTHERS => statements ]
END CASE ;
```


(Sequential also)

VHDL Dataflow statements

```
signal <= [ TRANSPORT ] value [AFTER time ];

subprogram( para1 [, para2 [ , ... ] ] );

[ ASSERT condition ] REPORT message-if-false
    [ SEVERITY level ] ;

level ::= note | warning | error | failure
```

(Dataflow only)

{} *meta-language grouping brackets - not VHDL*

```
signal <=  value WHEN condition
         [ ELSE value WHEN condition ]
           ELSE value ;

WITH sel SELECT
  signal <=  value WHEN sel-case
           [ , value WHEN sel-case ]
             [ , value WHEN OTHERS ] ;

label :FOR var IN range GENERATE
     dataflow-statements
END GENERATE;

IF condition GENERATE
     dataflow-statements
END GENERATE;

label : {ENTITY entity-name} | component-name
    GENERIC MAP( gen-map [ , gen-map ] )
    PORT MAP( port-map  [, port map] ) ;

[ label : ] PROCESS [ ( sensitivity-list ) ]
          process-declarations
    BEGIN sequential statements
    END PROCESS [ label ];

[ label : ] BLOCK local-declarations
    BEGIN dataflow-statements
    END [ label ] ;
```


Signal Attributes & Design Units

Signal Attributes

SIGNAL x has type T

Expression	Type	Description
x'EVENT	Boolean	TRUE if event on x
x'DELAYED(*del*)	T	x delayed by time *del*
x'LAST_VALUE	T	value of x before last or current event
x'LAST_EVENT	TIME	elapsed time from last event
x'STABLE(*tim*)	Boolean	FALSE if event on x within time *tim*

Design Units

```
ENTITY myentity IS
GENERIC( signal-list );
PORT( port-signal-list );
END myentity;

ARCHITECTURE archname OF entityname IS
declaration-statements
BEGIN
dataflow-statements
END archname;

PACKAGE mypackage IS
declarations
END [mypackage];

PACKAGE BODY mypackage IS
function and procedure body definitions
END PACKAGE BODY mypackage;
```


VHDL array syntax

```
unconstrained_array_type ::= STD_LOGIC_VECTOR | SIGNED | UNSIGNED | etc
range ::=   low TO high | high DOWNTO low |  array_signal 'RANGE

TYPE my_type IS ARRAY range OF base_type;
SUBTYPE my_subtype IS unconstrained_array_type( range ) ;

SIGNAL | VARIABLE | CONSTANT name : unconstrained_array_type( range );

my_array( range )   -- array slice on LHS or RHS
my_array( index )   -- array element on LHS or RHS
( val1, value2, value3) -- array value using element values specified via position on RHS
( index1=>val1, index2=>val2, ...., OTHERS=>valn ) -- array value on RHS

array'LEFT          array'LOW
array'RIGHT         array'HIGH
array'LENGTH
array'RANGE (see definitions of range above)
```

VHDL Declarations

Declarations

SIGNAL *sname* : *stype* [:= *init_val*];
CONSTANT *cname*: *ctype* := *init_val*;
VARIABLE vname: vtype [:= init_val];
SHARED VARIABLE vname: vtype [:= init_val];
FILE *fname*: *ftype* [OPEN *file_open_kind* IS *file_name_string*];
TYPE *tname* : *tspec*;

Types

INTEGER 1, 123, -3456
NATURAL <non-negative integer>
REAL 1.21, -0.033
CHARACTER '*', '0',
STRING "my string"
BOOLEAN FALSE, TRUE
TIME 10.1 ns, 11 fs, 10 min (units fs, ps, ns, us, ms, s, min, hr) -- physical time constant
INTEGER RANGE *low* TO *high* -- fixed range integer type
TYPE *enumeration_type* IS (*value_name-1*, *value_name-2*, ..., *value_name-n*); --enumeration type

VHDL Operators

Logical (**not** is unary)	**and**, **or**, **nand**, **nor**, **xor**, **xnor**, **not** S X S -> B, B X B -> B (unary S->S, B->B)
Relational	=,/= (any type) <,<=,>,<= (scalar or discrete types)
Shift	**sll**,**srl**,**sla**,**sra**,**rol**,**ror** V X I -> V Shift Left/Right Logical/Arithmetic Rotate Left/Right
Addition	+, - N->N, N X N -> N (all same type) Also in *numeric_std* for V X V -> V etc
Multiply	*,/ N X N->N (all same type) **mod**, **rem** Integer
Misc **: exponentiation **abs**: absolute value	** N X N->N (all same type) a ** b = a^b **abs** N->N (both same type)
Concatenation	& V X V -> V, V X S -> V, S X V -> V

Key to Types

V: std_logic_vector
N: integer or real
I: integer
S: std_logic
B: Boolean

Vector lengths a,b:

	length result
a + b	-> max(a,b)
a * b	-> a+b
a mod b	-> b

Logical, Relational, Shift, Additive, Concatenation ops are synthesisable on *vectors* and *fixed range integers*

VHDL Text File I/O

TYPE *file_type* IS FILE OF *element-type*;

FILE *file_object*: *file_type* OPEN *file_open_kind* IS *file_name_string*;

--"Automatic" file open & close in object declaration

--TEXT file access uses Package STD.TEXTIO

--Use statement required (but no library statement, since STD is always available)

--Defines TEXT file type, LINE linebuffer type

TYPE SIDE IS (right, left);

TYPE FILE_OPEN_KIND IS (read_mode, write_mode, append_mode);

TYPE FILE_OPEN_STATUS IS (open_ok, status_error, name_error, mode_error);

file_open(VARIABLE status: OUT FILE_OPEN_STATUS;
FILE *f*: TEXT: name: IN STRING;
mode: IN FILE_OPEN_KIND := read_mode);

file_close(FILE f: TEXT);

endfile(FILE f: TEXT) RETURN boolean;

readline(FILE *f*: TEXT; *line*: OUT LINE);
-- read the next line from f into line

writeline(FILE *f*: TEXT; *line*: INOUT: LINE);
-- write a line from line to f, clearing line

read(*line*: INOUT LINE; *value*: OUT <*any-type*>);

write(*line*: INOUT LINE;
value: IN <*any-type*>;
justified: IN SIDE := right;
field: IN WIDTH := 0);

<*any-type*> ::= BIT, BIT_VECTOR, CHARACTER, INTEGER, REAL, TIME, STRING

Functions & Procedures

Meta-language

[]	*optional part*
*	*0 or more repetitions*
{}	*grouping brackets*

```
PACKAGE mypack IS
   <function header>;
END PACKAGE mypack;
```

<function header> ::=
[IMPURE] FUNCTION *myfunc*[(<par> { ; <par> }*)]
RETURN *rtype*;
<par> ::= [<pspec>] *pname* : [<fmode>] *ptype* ;
<pspec> ::= FILE | SIGNAL | VARIABLE | CONSTANT
-- omit <pspec> for value IN parameter
-- may omit <pspec> for VARIABLE OUT mode
<fmode ::= IN | OUT | INOUT -- default mode is IN
-- functions can only have IN parameters

```
PACKAGE BODY mypack IS
   <function_header> IS
      <function-declarations>
   BEGIN
      <function-statements>
   END FUNCTION;
END PACKAGE BODY mypack;
```

<function-statement>::= <sequential_statement> |
RETURN *expression*;

- **PROCEDURE as for FUNCTION but**
  - **No RETURN *rtype* in header**
  - **WAIT allowed in body**
  - **OUT, INOUT parameters are allowed**
- **FUNCTIONs & PROCEDUREs can be defined in package body without duplicating header in package - in this case their scope is restricted to package body.**
- **Functions with zero parameters have no brackets in header or function call.**

Built-in functions

now -- returns current simulation time
'POS -- CHARACTER -> ASCII code
'VAL -- ASCII code -> CHARACTER

<scalar_type_name>'IMAGE(<value>)
-- <value> -> printable string

EE3-06
EE9 AC5

Answers 2012

Question 1.

The 3 parts of this question test basic VHDL writing & simulation skills & understanding.

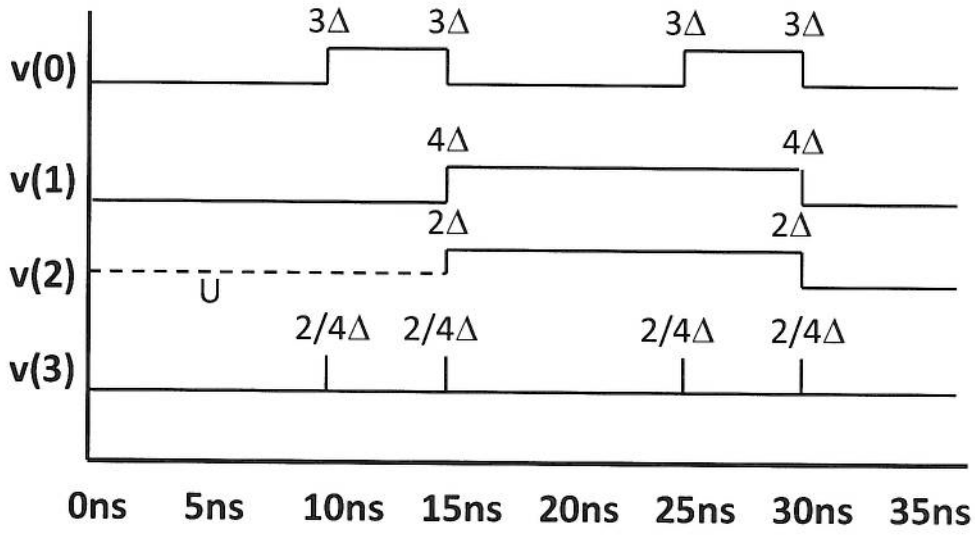
a)

marks will be deducted for less compact answers.

```
ARCHITECTURE synth OF counter IS
  SIGNAL q_int : unsigned(p'range);
BEGIN
  P1 : PROCESS
  BEGIN
    WAIT UNTIL clk'event AND clk = '0';
    IF reset = '1' OR q_int = unsigned(p) - 1 THEN
      q_int <= (OTHERS => '0');
    ELSE
      q_int <= unsigned(q_int) - 1;
    END IF;
  END PROCESS P1;
  q <= std_logic_vector(q_int);
END ARCHITECTURE synth;
```

[6 marks]

b)



[6 marks]

c)

```
ARCHITECTURE synth OF fsm IS

  TYPE   state IS (a, b, c);
  SIGNAL ss, ss_next : state;
BEGIN
  F1 : PROCESS
  BEGIN
    WAIT UNTIL clk'event AND clk = '1';
    IF reset = '1' THEN
      ss <= a;
    ELSE
      ss <= ss_next;
    END IF;
  END PROCESS F1;

  F2 : PROCESS(p, q, r, w, ss)
  BEGIN

    ss_next <= ss;
    match1 <= '0';
    match2 <= '1';
    done <= '0';
    CASE ss IS
      WHEN a =>
        IF signed(p) > signed(r) THEN
          ss_next <= b;
        END IF;
        IF q = p THEN
          match1 <= '1';
        END IF;
      WHEN b =>
        IF unsigned(q) = unsigned(w) + 1 THEN
          ss_next <= c;
        END IF;
      WHEN c =>
        ss_next <= a; done <= NOT reset; match2 <= '0';
    END CASE;
  END PROCESS F2;

END ARCHITECTURE synth;
```

[8 marks]

Question 2.

This question tests understanding of synthesis issues when writing code (part a), optimisation (parts b,d) and use of generics & concurrent loops (parts b,c).

a)
P1 - incomplete sensitivity list
will cause pre-synth simulation to be different from hardware (which completes the sensitivity ;list).

y1 - multiply driven P1/P2
Will cause X values in pre-synth simulation and may not synthesise. In real hardware result of contention is unpredictable and may cause excess supply current, but will otherwise be identical to pre-synthesis simulation where this has non-X values.

P0 comb process makes latch (incomplete IF)
Will cause possible hold time violations and mean that post-synth code does not reliably simulate as pre-synth code. Hardware will also be unpedictable due to possible hold time violations.

[6 marks]

b)

```
ENTITY compute1 IS
  GENERIC(n : integer);
  PORT(
    x           : IN  std_logic_vector(7 DOWNTO 0);
    y           : OUT std_logic_vector(7 DOWNTO 0);
    clk, reset : IN  std_logic);
END ENTITY compute1;

ARCHITECTURE struct1a OF xx IS
  SIGNAL a, b, c     : std_logic_vector(7 DOWNTO 0);
  SIGNAL clk, reset : std_logic;
BEGIN
  I1 : ENTITY compute1 GENERIC MAP (2)
           PORT MAP (x => a, y => b, clk => clk, reset => reset);
  I2 : ENTITY compute1 GENERIC MAP (3)
           PORT MAP (x => b, y => c, clk => clk, reset => reset);
END ARCHITECTURE struct1a;
```

[4 marks]

c)

```
ARCHITECTURE power_of_compute OF power IS
  TYPE   cc_type IS ARRAY (0 TO R) OF std_logic_vector(7 DOWNTO 0);
  SIGNAL cc          : cc_type;
BEGIN
  cc(0) <= a;
  b     <= cc(R);
  F1: FOR i IN 1 TO R GENERATE
    I1 : ENTITY compute1 PORT MAP (
      n     => i,
      x     => cc(i-1),
      y     => cc(i),
      clk   => clk,
      reset => reset);
  END GENERATE;
END ARCHITECTURE power_of_compute;
```

[8 marks]

d) *Compute1* could be used because the loop variable, which is connected to n in each iteration, is a compile-time constant after FOR GENERATE unrolling and hence will be valid as a generic parameter.

[2 marks]